

Projektowanie programu obiektowego

Przykład

System obsługi serwisu komputerowego

Opis systemu

Projekt zakłada stworzenie aplikacji wspomagającej pracę punktu serwisowego zajmującego się naprawą komputerów i urządzeń elektronicznych. System powinien umożliwiać rejestrowanie zgłoszeń serwisowych od klientów, przypisywanie ich do pracowników, śledzenie statusu naprawy oraz zarządzanie historią napraw. Każde zgłoszenie powinno zawierać podstawowe dane klienta, opis usterki, przewidywany czas naprawy oraz kosztorys.

W systemie powinny być obsługiwane różne etapy realizacji zlecenia – np. „przyjęte”, „w trakcie”, „oczekiwanie na części”, „zakończone”. Możliwe jest również przypisanie urządzenia do konkretnego typu (np. laptop, komputer stacjonarny, drukarka), a także powiązanie kilku urządzeń z jednym klientem.

1. Analiza wymagań

Pierwszy etap tworzenia systemu informatycznego, polegający na zebraniu, zrozumieniu i uszczegółowieniu potrzeb użytkowników oraz funkcji, które system ma realizować.

- Określamy co system ma robić (zanim zaczniemy myśleć jak)
- Zbieramy informacje z opisu projektu (rozmów z klientem, analizy istniejących rozwiązań)
- Spisujemy funkcjonalności, modelujemy scenariusze działania

1. **Rejestracja zgłoszeń serwisowych** – przyjmowanie informacji o kliencie, urządzeniu i opisie usterki.
2. **Zarządzanie klientami** – dane osobowe, kontakt.
3. **Zarządzanie urządzeniami** – różne typy: laptop, komputer stacjonarny, drukarka itp.; jeden klient może mieć wiele urządzeń.
4. **Status naprawy** – etapy realizacji: Przyjęte , W trakcie , Oczekiwanie na części , Zakończone .
5. **Przypisywanie pracowników** – przydzielanie zleceń konkretnym technikom.
6. **Historia napraw** – zapis przebiegu i kosztów.

2. Identyfikacja klas domenowych

Kryteria wyróżniania klas

- pojedyncza odpowiedzialność – każda klasa powinna mieć jedną dobrze zdefiniowaną rolę.
- trwałość stanu – byty, których stan przechowujemy w bazie (np. zgłoszenie, klient).
- zmienna liczba instancji – encje, które mogą występować wielokrotnie (klientów jest wielu, zgłoszeń jest wiele).

Na tej podstawie wyodrębniamy podstawowe byty:

- Customer (Klient) — przechowuje dane klienta;
- Device (Urządzenie) — reprezentuje fizyczne urządzenie;
- ServiceRequest (Zgłoszenie serwisowe) — pojedyncze zgłoszenie do naprawy;
- Employee (Pracownik) — osoba realizująca naprawę;
- Status / DeviceType — wartości słownikowe (enum) do etapów realizacji i typów urządzeń.

 Customer

 Device

 ServiceRequest

 Employee

 Status

 DeviceType

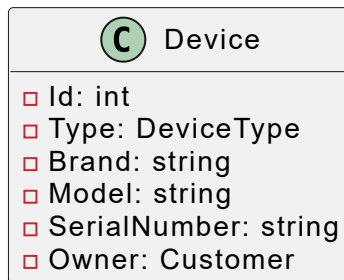
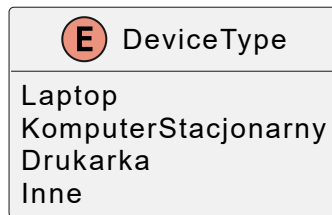
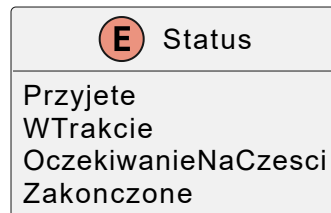
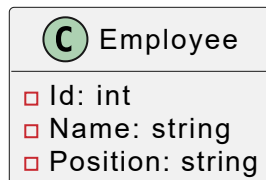
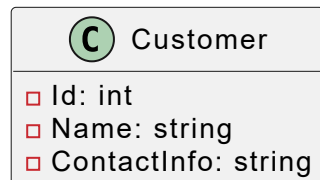
3. Wybór atrybutów klas

Każdy byt ma dane, które opisują jego stan i są niezbędne do realizacji wymagań

- **Customer:** unikalne `Id` (identyfikacja), `Name`, `ContactInfo` (do powiadomień).
- **Device:** `Id`, `Type` (enum `DeviceType`), marka/model, `SerialNumber` (śledzenie), referencja do właściciela (`Owner`).
- **ServiceRequest:** `Id`, referencje do `Customer` i `Device`, `Description` (opis usterki), `EstimatedTime`, `CostEstimate`, `Status`, `AssignedEmployee`, `CreatedAt`, `History` (lista wpisów opisujących przebieg).
- **Employee:** `Id`, `Name`, `Position`.

Zasada: uwzględniamy tylko dane potrzebne do implementacji funkcji; nie gromadzimy nadmiarowych pól.

Diagram klas



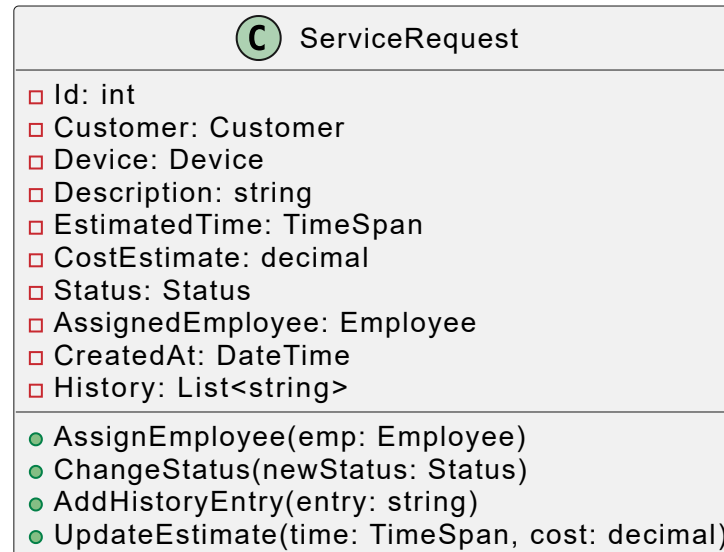
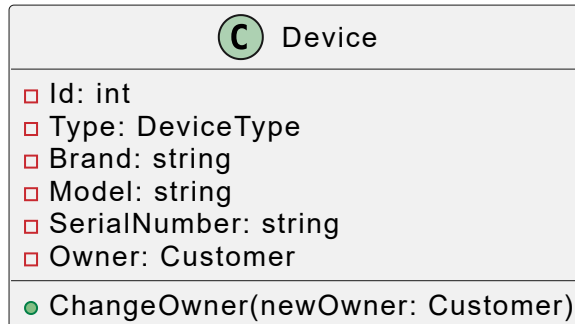
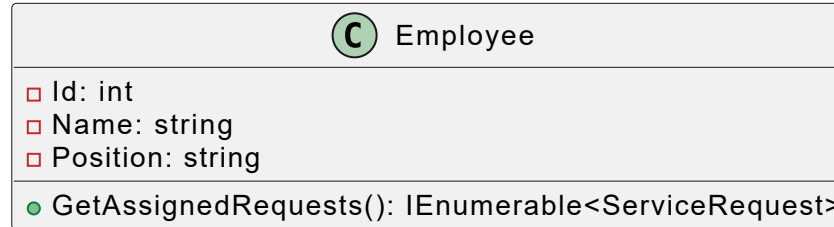
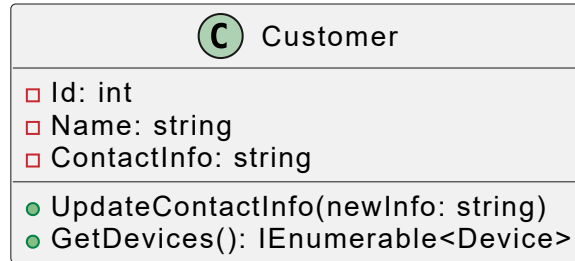
4. Wybór metod klas

Metody odpowiadają na pytanie: jakie działania będziemy wykonywać na tym bycie?

- **Customer:** `UpdateContactInfo(...)` — zmiana danych kontaktowych; `GetDevices()` — pobranie urządzeń tego klienta.
- **Device:** `ChangeOwner(...)` — przekazanie urządzenia innemu klientowi.
- **ServiceRequest:**
 - `AssignEmployee(...)`, `ChangeStatus(...)` — zmiana stanu zlecenia;
 - `AddHistoryEntry(...)` — doklejanie kolejnych kroków do historii;
 - `UpdateEstimate(...)` — korekta czasu/kosztu.
- **Employee:** `GetAssignedRequests()` — lista zleceń przypisanych do pracownika.

Zasada: metody odzwierciedlają typowe interakcje użytkownika i zmiany stanu.

Diagram klas

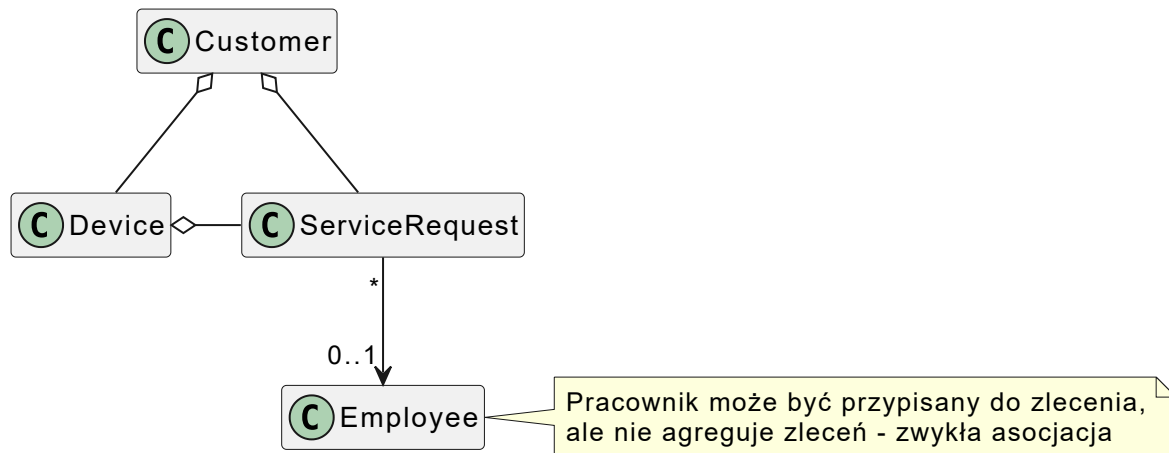


5. Identyfikacja relacji między klasami

Opis słowny często na nie wskazuje: „klient może mieć wiele urządzeń”, “jedno urządzenie może mieć wiele zgłoszeń”.

1. Szukamy fraz: „jeden ... może mieć wiele ...”, „wiele ... należy do jednego ...”;
2. Określamy kierunek i kardynalność (1-do-1, 1-do-wielu, wiele-do-wielu, opcjonalność – np. `AssignedEmployee` może być null przed przydzieleniem).

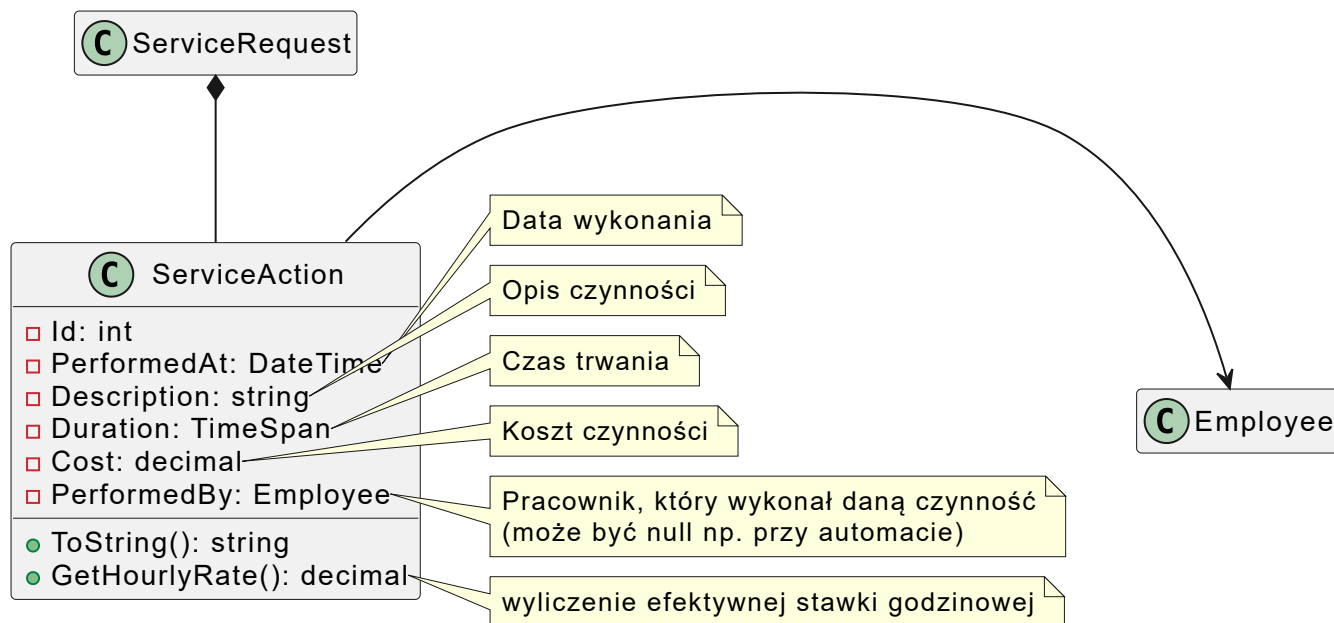
- `Customer` 1—* `Device` (jeden klient może mieć wiele urządzeń)
- `Customer` 1—* `ServiceRequest` (jeden klient może mieć wiele zgłoszeń)
- `Device` 1—* `ServiceRequest` (jedno urządzenie może mieć wiele zgłoszeń na przestrzeni czasu)
- `Employee` 1—* `ServiceRequest` (jeden pracownik może być przypisany do wielu zgłoszeń)



Modelowanie innych relacji

Możemy rozważyć wprowadzenie czynności serwisowej (np. „wymiana dysku”, „czyszczenie chłodzenia”), która pozwoli dokładniej śledzić co konkretnie zostało wykonane w ramach naprawy.

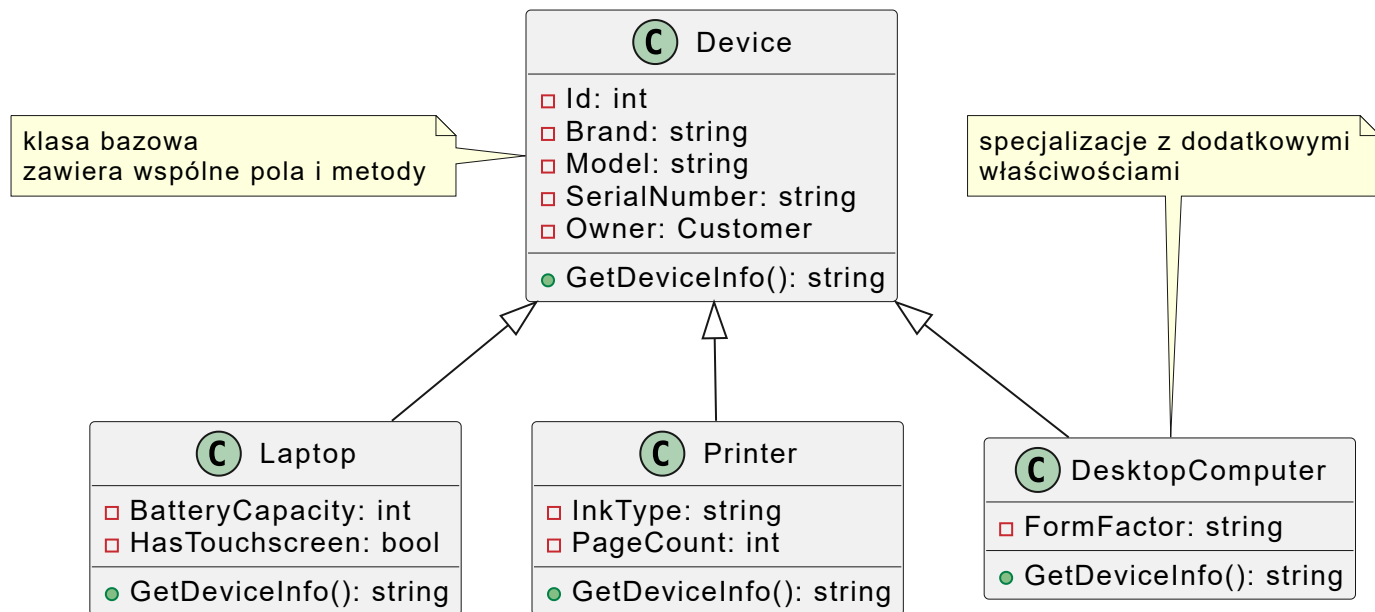
- Jednostkowa akcja wykonana przez pracownika w ramach zgłoszenia serwisowego
- Relacja kompozycji (silna) – bez zgłoszenia nie ma sensu istnienie czynności
- Każde zgłoszenie może mieć wiele czynności



Dziedziczenie

Gdzie można zastosować dziedziczenie?

- Możemy stworzyć specjalizacje różnych typów sprzętów
- Ma to sens, jeśli będą mieć one **dodatkowe atrybuty** lub **różne zachowania**
- Niektóre urządzenia mogą wymagać innych czynności serwisowych

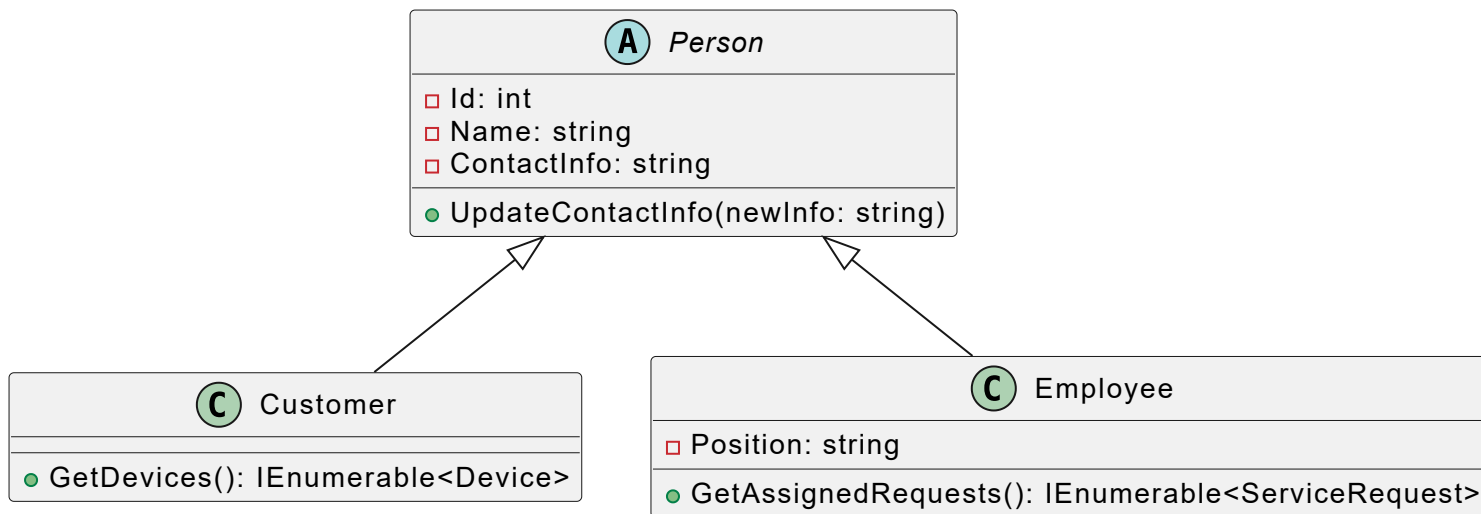


- Nie warto dziedziczyć „na siłę”, jeśli klasy mają tylko pozornie podobne dane, ale inny cykl życia i żadnych wspólnych zachowań.

Dziedziczenie

Dziedziczenia dla generalizacji

- Wyodrębniamy wspólną klasę bazową dla bytów, które mają część cech wspólnych, ale pełnią różne role
- `Customer` i `Employee` – obie klasy reprezentują osobę, mają `Id`, `Name`, `ContactInfo`.
- Mają wspólne zachowania (np. aktualizacja danych kontaktowych),
- Możemy je uogólnić do `Person` (wprowadzamy klasę bazową i dziedziczymy z niej role specjalizowane).



- Możliwość rozszerzenia w przyszłości – np. różne typy klientów (prywatny i firmowy), pracowników (freelancer) lub urządzeń (elektronika użytkowa vs. AV)

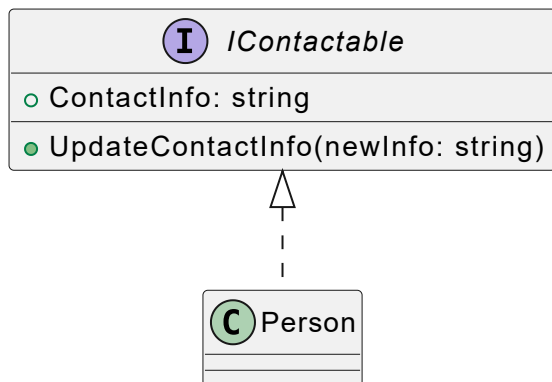
Interfejsy

Wprowadzenie interfejsów do projektu zwiększy elastyczność, czytelność i testowalność kodu

- Mamy kilka klas, które posiadają dane kontaktowe i mogą je aktualizować: `Customer` i `Employee`
- Warto wyodrębnić wspólny kontrakt:

```
1 public interface IContactable {  
2     string ContactInfo { get; set; }  
3     void UpdateContactInfo(string newInfo);  
4 }
```

- Klasa `Person` implementuje interfejs



- Możemy teraz stworzyć metodę przyjmującą dowolne `IContactable` jako parametr:

```
1 void Notify(IContactable person) { ... }
```

Polimorfizm i metody wirtualne

Ten mechanizm pozwala na definiowanie domyślnego zachowania, które może być nadpisane przez klasy dziedziczące

- Klasa `Device` oferuje metodę `GetDeviceInfo()`, która jest realizowana inaczej w każdej w klas dziedziczących
- Metoda `Person.UpdateContactInfo(...)` może być inaczej implementowana w każdej klasie, jeśli różne osoby mają inne zasady walidacji lub powiadamiania.
- Jeśli system ma obsługiwać zarówno naprawy, jak i sprzedaż części, to dobrym pomysłem będzie wprowadzenie wspólnej klasy bazowej dla: naprawy (`ServiceRequest`) i zamówienia części (`PartOrder`)

